

dbt(Data Build Tool) Tutorial

https://www.youtube.com/watch?v=XWyl_IS38ME

Introduction

- Difference between ETL and ELT
- What is DBT
- What is DBT Cloud
- Essential steps in the development of data pipelines
- Need for ELT templatzation
- Pre-requisites, installing DBT and configuring a DBT project.
- Discussion on supporting databases
- Creating a Snowflake account and creating needed database objects
- Introduction to Jinja

Project files and folders

- Familiarizing with existing files and folders
- Compiler, Runner, Package manager
- Communication between the files and folders
- Configure and run a simple job

Understanding git

- Whisare git desktop, git server, GitHub and GitLab
- Download, Install, and configure the git for the DBT directory
- Practicing add, status, commit, and git log
- Working with branches
- Working with git on-premise
- Understanding and working with GitHub
- practicing add, push, and pull requests for GitHub

Working with Models

- Types of materialization
 - Table
 - View
 - Incremental

- Ephemeral

Working with configurations
Working with variables
Running the models by giving the values for the variables

Performing data testing

- Advantages of the Test feature
- Working with basic tests and tests folder
- Unique, not null, and relationship/or references
- Running the DBT tests
- Writing the custom tests

- DBT is used for transforming raw data into analytics-ready data by applying a series of [SQL](#)-based transformations.
- It's used for creating and managing data pipelines, which can be tested, documented, and version-controlled.
- DBT can be used to build complex data models, which can be reused across multiple projects.
- It's used to automate data transformation workflows, reducing manual errors and increasing efficiency.
- DBT is used to collaborate on data projects between data analysts, data engineers, and other stakeholders.
- updating earlier data and retrieving information from the database.
- It can be used to implement best practices for data transformation, such as incremental loading and data lineage tracking.
- DBT is used to improve data quality, consistency, and accuracy, which leads to better business insights and decision-making.

What is DBT?

- DBT is a software tool designed specifically for data engineers and analysts to transform and process data efficiently.
- DBT helps build data pipelines by providing a code-based approach to defining and executing data transformations.
- DBT uses SQL (Structured Query Language) as its primary language for defining transformations, making it accessible to those familiar with SQL.
- DBT promotes modularity by breaking down complex transformations into smaller, reusable components called models, making it easier to manage and maintain data pipelines.
- DBT provides built-in testing capabilities for ensuring the quality and accuracy of data transformations. It also allows for documenting those processes, which eases collaboration on these workflows.

- DBT can work with a variety of data warehouses, including Snowflake, BigQuery, and Redshift.

DBT certification training course:

- Fundamentals of Data Built Tool
- Why do we use DBT?
- Configuring DBT locally and in the cloud
- Connect DBT to a Database
- Creation of SQL transformations
- Testing underlying data and SQL transformation
- Scheduling transformations
- Testing code in a dev environment
- Data built tool best practices
- What is DBT
- Typical data transformation challenges
- How does DBT eliminate traditional Data challenges?
- Data Built Tool and databases
- Version Control
- Data loading into the data warehouse
- Data repository set up & connect to DBT cloud
- Fundamentals of DBT cloud IDE
- The process to connect to Repository
- The process to connect to your warehouse

Models Topics:

- Overview of Models and types
- The process to build a model
- Basics of modularity
- Ref functions
- Data modeling fundamentals
- Naming conventions
- Project reorganization

Tests Topics:

- What is testing?
- Need of Testing
- Data tests
- Schema tests
- Live test scenarios

Documentation Topics:

- Fundamentals of Documentation
- Importance of documentation
- The process to write doc blocks
- Documentation writing
- Documentation generation and view
- Working with real-time documentation & gaining hands-on experience.

Sources Topics:

- Introduction to Sources
- Source Configuration
- Selecting from sources
- Testing Sources
- Documenting Sources
- Source Freshness
- Working with real-time source scenarios.

Deployment Topics:

- What is Deployment
- The process to schedule a job in the DBT cloud
- Reviewing scheduled jobs.

1. Introduction

2. Dbt, the T in ELT

3. Project

- 3.1. Project Demo
 - Prerequisites
- 3.2. Configurations and connections
 - 3.2.1. profiles.yml

- [3.2.2. dbt_project.yml](#)
- [3.3 Data flow](#)
 - [3.3.1. Source](#)
 - [3.3.2. Snapshots](#)
 - [3.3.3. Staging](#)
 - [3.3.4. Marts](#)
 - [3.3.4.1. Core](#)
 - [3.3.4.2. Marketing](#)
- [3.4. dbt run](#)
- [3.5. dbt test](#)
- [3.6. dbt docs](#)
- [3.7. Scheduling](#)
- [4. Conclusion](#)
- [5. Further reading](#)
- [6. References](#)

1. Introduction

If you are a student, analyst, engineer, or anyone in the data space and are curious about what **dbt** is and how to use it. Then this post is for you.

If you are keen to understand why dbt is widely used, please read [this article](#) .

2. Dbt, the T in ELT

In an ELT pipeline, the raw data is loaded(EL) into the data warehouse. Then the raw data is transformed into usable tables, using SQL queries run on the data warehouse.

Note: If you are interested in learning to write efficient SQL for data processing, checkout my e-book: [Efficient Data Processing in SQL](#)

dbt provides an easy way to create, transform, and validate the data within a data warehouse. **dbt** does the T in ELT (Extract, Load, Transform) processes.

In dbt, we work with **models, which is a sql file with a select statement**. These models can depend on other models, have tests defined on them, and can be created as tables or views. The names of models created by dbt are their file names. dbt uses the file names and its references in other files(aka models) to automatically define the data flow.

E.g. The file [dim_customers.sql](#) represents the model named `dim_customers`. This model depends on the models [stg_eltool_customers](#) and [stg_eltool_state](#) .

The `dim_customers` model can then be referenced in other model definitions.

```
with customers as (  
    select *  
    from {{ ref('stg_eltool__customers') }}  
)  
state as (  
    select *  
    from {{ ref('stg_eltool__state') }}  
)  
select c.customer_id,  
       c.zipcode,  
       c.city,  
       c.state_code,  
       s.state_name,  
       c.datetime_created,  
       c.datetime_updated,  
       c.dbt_valid_from::TIMESTAMP as valid_from,  
       CASE  
           WHEN c.dbt_valid_to IS NULL THEN '9999-12-31'::TIMESTAMP  
           ELSE c.dbt_valid_to::TIMESTAMP  
       END as valid_to  
from customers c  
    join state s on c.state_code = s.state_code
```

We can **define tests to be run on processed data** using dbt. Dbt allows us to create 2 types of tests, they are

1. **Generic tests**: Unique, not_null, accepted_values, and relationships tests per column defined in YAML files. E.g. see [core.yml](#)

2. **Bespoke (aka one-off) tests**: Sql scripts created under the **tests** folder. They can be any query. They are successful if the sql scripts do not return any rows, else unsuccessful.

E.g. The **core.yml** file contains tests for **dim_customers** and **fct_orders** models.

```
version: 2

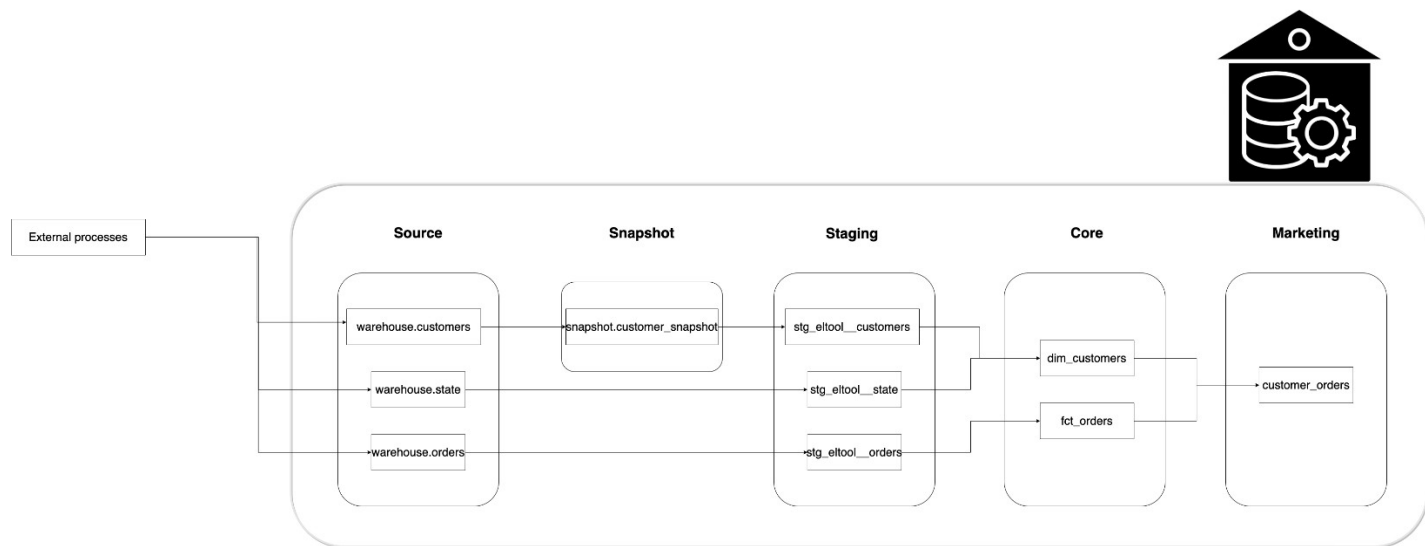
models:
  - name: dim_customers
    columns:
      - name: customer_id
        tests:
          - not_null # checks if customer_id column in
dim_customers is not null
  - name: fct_orders
```

3. Project

We are asked by the marketing team to create a denormalized table **customer_orders**, with information about every order placed by the customers. Let's assume the **customers** and **orders** data are loaded into the warehouse by a process.

The process used to bring these data into our data warehouse is the EL part. This can be done using a vendor service like [Fivetran](#) , [Stitch](#) , or open-source services like [Singer](#) , [Airbyte](#) or using a custom service.

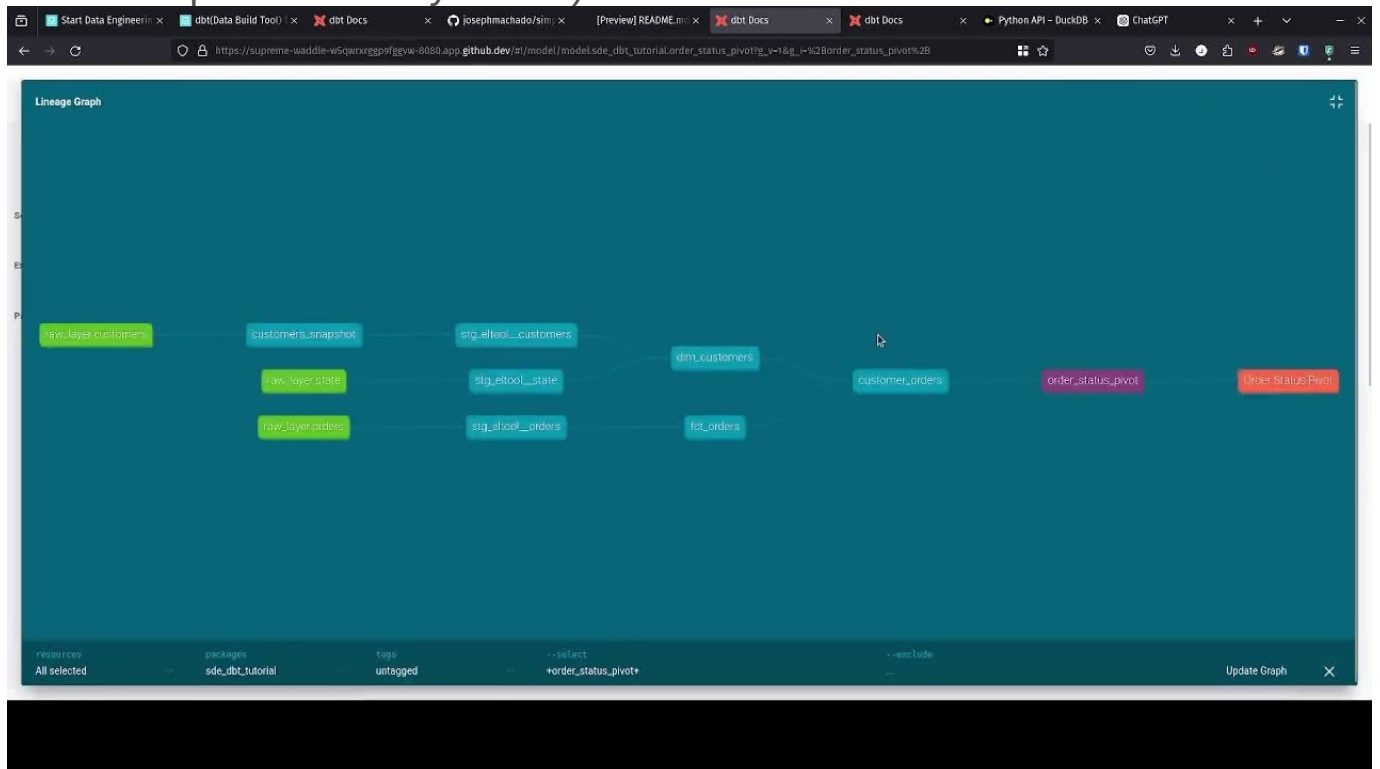
Let's see how our data is transformed into the final denormalized table.



We will follow data warehouse best practices like having [staging tables](#) , testing, using [slowly changing dimensions type 2](#) and naming conventions.

3.1. Project Demo

Here is a demo of how to run this on CodeSpaces(click on the image below to open video on youtube):



Prerequisites

1. [python ^3.11](#)
2. [git](#)

Clone the git repo as shown below:

```
git clone https://github.com/josephmachado/simple_dbt_project.git
cd simple_dbt_project
```

Setup python virtual environment as shown below:

```
rm -rf myenv
# set up venv and run dbt
python -m venv myenv
source myenv/bin/activate
pip install -r requirements.txt
```

Run dbt commands as shown below:

```
dbt clean
dbt deps
dbt snapshot
```

```
dbt run
dbt test
dbt docs generate
dbt docs serve
```

Go to <http://localhost:8080> to see the dbt documentation. If you are running this on GitHub CodeSpaces the port 8080 will be open automatically. Press Ctrl + c to stop the document server.

In our project folder you will see the following folders.

```
.
├── analysis
├── data
├── macros
├── models
│   ├── marts
│   │   ├── core
│   │   └── marketing
│   └── staging
├── snapshots
└── tests
```

1. **analysis**: Any **.sql** files found in this folder will be compiled to raw sql when you run **dbt compile**. They will not be run by dbt but can be copied into any tool of choice.
2. **data**: We can store raw data that we want to be loaded into our data warehouse. This is typically used to store small mapping data.
3. **macros**: Dbt allows users to create macros, which are sql based functions. These macros can be reused across our project.

We will go over the **models**, **snapshots**, and **tests** folders in the below sections.

Note: The project repository has advanced features which are explained in the [uplevel dbt workflow article](#) . It is recommended to read this tutorial first before diving into the advanced features specified in the [uplevel dbt workflow article](#) article.

3.2. Configurations and connections

Let's set the warehouse connections and project settings.

3.2.1. profiles.yml

Dbt requires a `profiles.yml` file to contain data warehouse connection details. We have defined the warehouse connection details at `./profiles.yml`.

The `target` variable defines the environment. The default is dev. We can have multiple targets, which can be specified when running `dbt` commands.

The profile is `sde_dbt_tutorial`. The `profiles.yml` file can contain multiple profiles for when you have more than one dbt project.

3.2.2. dbt_project.yml

In this file, you can define the profile to be used and the paths for different types of files (see `*-paths`).

Materialization is a variable that controls how dbt creates a model. By default, every model will be a view. This can be overridden in `dbt_project.yml`. We have set the models under `models/marts/core/` to materialize as tables.

```
# Configuring models
models:
  sde_dbt_tutorial:
    # Applies to all files under models/marts/core/
    marts:
      core:
        materialized: table
```

3.3 Data flow

We will see how the `customer_orders` table is created from the source tables. These transformations follow warehouse and dbt best practices.

3.3.1. Source

Source tables refer to tables loaded into the warehouse by an EL process. Since dbt did not create them, we have to define them. This definition enables referring to the source tables using the [source](#) function. For e.g. `{{ source('warehouse', 'orders') }}` refers to the `warehouse.orders` table. We can also define tests to ensure that the source data is clean.

- Source definition: [/models/staging/src_eltool.yml](#)
- Test definitions: [/models/staging/src_eltool.yml](#)

3.3.2. Snapshots

A business entity's attributes change over time. These changes should be captured in our data warehouse. E.g. a user may move to a new address. This is called slowly changing dimensions, in data warehouse modeling.

Read [this article](#) to understand the importance of storing historical data changes, and what slowly changing dimensions are.

Dbt allows us to easily create these slowly changing dimension tables (type 2) using the snapshot feature. When creating a snapshot, we need to define the database, schema, strategy, and columns to identify row updates.

```
dbt snapshot
# alternative run the command 'just snapshot'
```

dbt creates a snapshot table on the first run, and on consecutive runs will check for changed values and update older rows. We simulate this as shown below

```
# Remove header from ./raw_data/customers_new.csv
# and append it to ./raw_data/customers.csv
echo "" >> ./raw_data/customers.csv
tail -n +2 ./raw_data/customer_new.csv >>
./raw_data/customers.csv
```

Run the snapshot command again

```
dbt snapshot
```

Raw data (raw_layer.customer) & Snapshot table (snapshots.customers_snapshot)

```
3 t/~/c/s/11/u/b/zsh 4
* H - Toggle database details
* r - Rename/Edit buffer/connection/save
* q - Close drawer
* <C-j>/<C-k> - Go to last/first sibling
* K/O - Go to prev/next sibling
* <C-p>/<C-n> - Go to parent/child node
* <Leader>W - (sql) Save currently opened
* <Leader>E - (sql) Edit bind parameters
* <Leader>S - (sql) Execute query in viz
* <C-j> - (.dbout) Go to entry from fore
* <motion>ic - (.dbout) Operator pending
* <Leader>R - (.dbout) Toggle expanded v

▼ dbt.duckdb ✓
+ New query
▼ Buffers (1)
  ► query-2024-05-31-15-38
▼ Saved queries (0)
▼ Tables (8)
  ► customer_orders
  ► fct_orders
  ► stg_eltool__orders
  ► customers_snapshot
  ► order_status_pivot
  ► stg_eltool__state
  ► dim_customers
  ► stg_eltool__customers

► Query results (1)
dbt1 Normal /tmp/nvim.josephkevinmachado@IADMH/dbtduckdb-query-2024-05-31-15-38-35 1:60
customer_id zipcode city state_code datetime_created datetime_updated dbt_scd_id dbt_updated_at dbt_valid_from dbt_valid_to
-----
82 59655 areia branca RN 2017-10-18 00:00:00 2017-10-18 00:00:00 d5ce49419bd8ed844a90e39c381a96d3 2017-10-18 00:00:00 2017-10-18 00:00:00 2017-10-18 00:10:00
82 24120 niteroi RJ 2017-10-18 00:00:00 2017-10-18 00:10:00 795e83e8b873e55d2443643dae7b1db0 2017-10-18 00:10:00 2017-10-18 00:10:00
```

The row with zipcode 59655 had its `dbt_valid_to` column updated. The `dbt from` and `to` columns represent the time range when the data in that row is representative of customer 82.

- Model definition: [/snapshots/customers.sql](#)

3.3.3. Staging

The staging area is where **raw data is cast into correct data types, given consistent column names, and prepared to be transformed into models used by end-users.**

You might have noticed the `eltool` in the staging model names. If we use Fivetran to EL data, our models will be named `stg_fivetran__orders` and the YAML file will be `stg_fivetran.yml`.

In `stg_eltool__customers.sql` we use the `ref` function instead of the `source` function because this model is derived from the `snapshot` model. In dbt, we can use the `ref` function to refer to any models created by dbt.

- Test definitions: [/models/staging/stg_eltool.yml](#)
- Model definitions: [/models/staging/stg_eltool_customers.sql ,stg_eltool_orders.sql ,stg_eltool_state.sql](#)

3.3.4. Marts

Marts consist of the core tables for end-users and business vertical-specific tables. In our example, we have a marketing department-specific folder to defined the model requested by marketing.

3.3.4.1. Core

The core defines the fact and dimension models to be used by end-users. The fact and dimension models are materialized as tables, for performance on frequent use. The fact and dimension models are based on [kimball dimensional model](#) .

- Test definitions: [/models/marts/core/core.yml](#)
- Model definitions: [/models/staging/dim_customers ,fct_orders.sql](#)

Dbt offers four generic tests, unique, not_null, accepted_values, and relationships. We can create one-off (aka bespoke) tests under the **Tests** folder. Let's create a sql test script that checks if any of the customer rows were duplicated or missed. If the query returns one or more records, the tests will fail. Understanding this script is left as an exercise for the reader.

- One-off test: [/tests/assert_customer_dimension_has_no_row_loss.sql](#)

3.3.4.2. Marketing

In this section, we define the models for **marketing** end users. A project can have multiple business verticals. Having one folder per business vertical provides an easy way to organize the models.

- Test definitions: [/models/marts/marketing/marketing.yml](#)

- Model definitions: [/models/marts/marketing/customer_orders.sql](#)

3.4. dbt run

We have the necessary model definitions in place. Let's create the models.

```
dbt snapshot # just snapshot
dbt run
# Finished running 5 view models, 2 table models, 2 hooks in 0
hours 0 minutes and 3.22 seconds (3.22s).
```

The `stg_eltool__customers` model requires `snapshots.customers_snapshot` model. But snapshots are not created on `dbt run`, so we run `dbt snapshot` first.

Our staging and marketing models are as materialized views, and the two core models are materialized as tables.

The snapshot command should be executed independently from the run command to keep snapshot tables up to date. If snapshot tables are stale, the models will be incorrect. There is snapshot freshness monitoring in [dbt cloud UI](#).

3.5. dbt test

With the models defined, we can run tests on them. Note that, unlike standard testing, these tests run after the data has been processed. You can run tests as shown below.

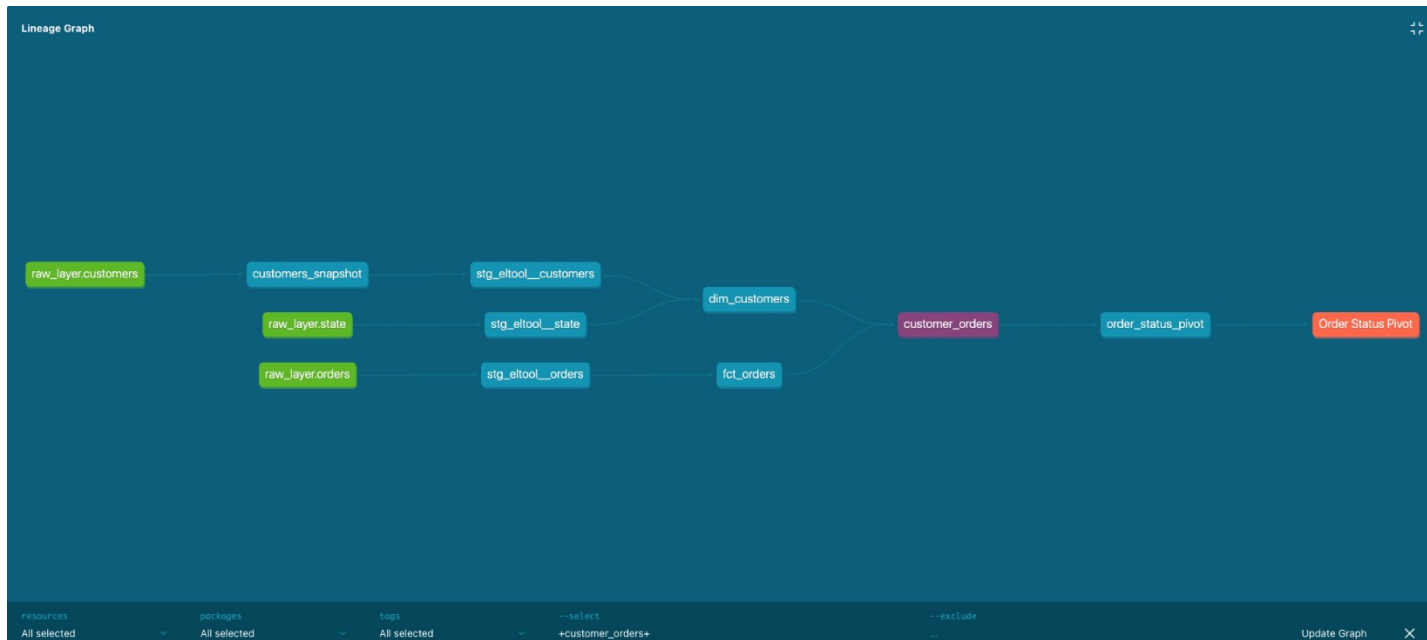
```
dbt test # or run "just test"
# Finished running 14 tests...
```

3.6. dbt docs

One of the powerful features of dbt is its docs. To generate documentation and serve them, run the following commands:

```
dbt docs generate
dbt docs serve
```

You can visit <http://localhost:8080> to see the documentation. Navigate to `customer_orders` within the `sde_dbt_tutorial` project in the left pane. Click on the view lineage graph icon on the lower right side. The lineage graph shows the dependencies of a model. You can also see the tests defined, descriptions (set in the corresponding YAML file), and the compiled sql statements.



3.7. Scheduling

We have seen how to create snapshots, models, run tests and generate documentation. These are all commands run via the cli. Dbt compiles the models into sql queries under the `target` folder (not part of git repo) and executes them on the data warehouse.

To schedule dbt runs, snapshots, and tests we need to use a scheduler. Dbt cloud is a great option to do easy scheduling. Checkout [this article](#) to learn how to schedule jobs with dbt cloud. The dbt commands can be run by other popular schedulers like cron, Airflow, Dagster, etc.

4. Conclusion

Dbt is a great choice to build your ELT pipelines. Combining data warehouse best practices, testing, documentation, ease of use, [data CI/CD](#) , [community support](#) and a great cloud offering, dbt has set itself up as an essential tool for data engineers. **Learning and understanding dbt can significantly improve your odds of landing a DE job** as well.

To recap, we went over

1. Dbt project structure
2. Setting up connections
3. Generating SCD2 (aka snapshots) with dbt
4. Generating models following best practices
5. Testing models
6. Generating and viewing documentation

dbt can help you make your ELT pipelines stable and development fun. If you have any questions or comments, please leave them in the comment section below.

5. Further reading

1. [dbt CI/CD](#)
2. [Why use dbt?](#)
3. [ETL v ELT](#)
4. [Unit test SQL in dbt](#)
5. [Simple data platform with Stitch and dbt](#)

6. References

1. [dbt snapshot best practices](#)
2. [dbt project structure](#)

As the maintainers of dbt, and analytics consultants, at [Fishtown Analytics](#) 529 we build a lot of dbt projects. Over time, we've developed internal conventions on how we structure them.

This article does not seek to instruct you on how to design a final model for your stakeholders — it won't cover whether you should denormalize everything into one wide master table, or have many tables that need to be joined together in the BI layer. There

are entire books on this topic. Instead, use this as a guide once you've already got an idea of what you're building for how you should break the transformations up into separate dbt models.

It's important to note that **this is not the only, or the objectively best, way to structure a dbt project**. Rather, this document reflects our current opinions. These opinions are strongly influenced by:

- our views on data model design; which in turn are influenced by:
- the kinds of analytics problems we are solving for clients
- the data stack we typically work within, in which multiple data sources are loaded by third party tools, and the data warehouse is optimized for analytical queries (therefore we aren't tightly bounded by performance optimization considerations).

Our opinions are **almost guaranteed to change over time** as we update our views on modeling, are exposed to more analytics problems, and data stacks evolve. It's also worth clearly stating here: the way we structure dbt projects makes sense for our projects, but may not be the best fit for yours! This article exists on Discourse so that we can have a conversation – I would love to know how others in the community are structuring their projects.

In comparison, the (recently updated) [best practices 3.8k](#) reflect principles that we believe to be true for any dbt project. Of course, these two documents go hand in hand – our projects are structured in such a way that makes the those principles easy to observe, in particular:

- Limit references to raw data
- Rename and recast fields once
- Group your models in directories
- Add tests to your models
- Consider the information architecture of your data warehouse
- Separate source-centric and business-centric transformations

If you want to see what the code for one of our projects looks like, check out [this demonstration dbt project 1.2k](#).

We also recently held (and recorded) an office hours on this topic – this article provides a high level outline, but there's a lot more detail and discussion in the [video 748](#).

Lastly, before I dive in, a huge thank you to [@jerco](#) for not only teaching me a lot of the material in this article, but also for doing a lot of the groundwork that went into this article – entire sections of this article are in fact lifted from his work.

Data transformation 101

The data in any of our projects has three distinct checkpoints:

1. **Sources:** Schemas and tables in a source-conformed structure (i.e. tables and columns in a structure based on what an API returns), loaded by a third party tool.
2. **Staging models:** The atomic unit of data modeling. Each model bears a one-to-one relationship with the source data table it represents. It has the same granularity, but the columns have been renamed, recast, or usefully reconsidered into a consistent format.
3. **Marts models:** Models that represent business processes and entities, abstracted from the data sources that they are based on.

In a simple project, these may be the only models you build; more complex projects may have a number of intermediate models that help along this journey, as well as accessories to these models (see below).

Still confused? An example might help! Let's think about a software business that uses both Stripe and Braintree to collect subscription payments. Their three stages of modeling might look like:

1. **Sources:** Payment records from the Stripe API and payment records from the Braintree API, loaded into their data warehouse by a third party tool.
2. **Staging models:** Both the Stripe and Braintree payments are recast into a consistent shape, with consistent column names.
3. **Marts models:** A monthly recurring revenue (MRR) model that classifies revenue per customer per month as new revenue, upgrades, downgrades, and churn, to understand how a business is performing over time. It may be useful to note whether the revenue was collected via Stripe or Braintree, but they are not fundamentally separate models.

Of note here is that there is a distinct change that occurs between the staging and marts checkpoints – sources and staging models are source-centric, whereas marts models are business-centric.

In our dbt projects, this leads us to our first split in our models/ directory which helps us make this distinction:

```
├─ dbt_project.yml
├─ models
│   └─ marts
│   └─ staging
```

Staging raw data

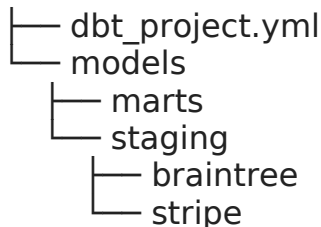
The goal of the staging layer is to create staging models. Staging models take raw data, and clean and prepare them for further analysis. For a user querying the data warehouse, a relation with a `stg_` prefix indicates that:

- Fields have been renamed and recast in a consistent way.¹

- Datatypes, such as timezones, are consistent.
- Light cleansing, such as replacing empty string with NULL values, has occurred.
- If useful, flattening of objects might have occurred.
- There is a primary key that is both unique and not null (and tested).

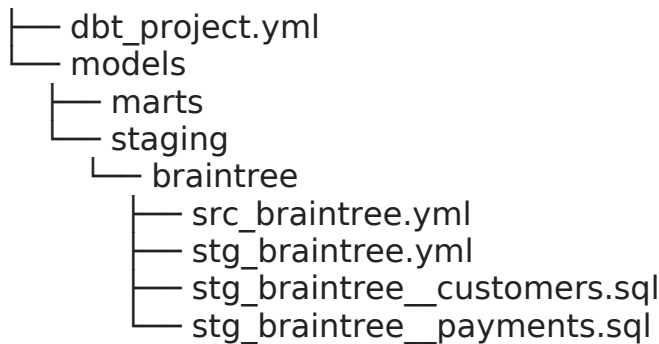
Staging models can have joins in them to field additional columns for context or enrichment; add rows through unions and remove them through filters; deduplicate a natural key or hash together a [surrogate one 352](#).

Because we often work with multiple data sources, in our staging directory, we create one directory per source.



Each staging directory contains at a minimum:

- One staging model for each object that is useful for analytics:
 - o Named `stg_<source>__<object>`.
 - o Generally materialized as a view (unless performance requires it as a table).
- A `src_<source>.yml` file which contains:
 - o [Source 1.3k](#) definitions, tests, and documentation
- A `stg_<source>.yml` file which contains
 - o [Tests and documentation 658](#) for models in the same directory



Some dbt users prefer to have one `.yml` file per model (e.g. `stg_braintree__customers.yml`). This is a completely reasonable choice, and we recommend implementing it if your `.yml` files start to become unwieldy.

But what about base models?

Earlier versions of the dbt documentation recommended implementing “base models” as the first layer of transformation – and we used to organize and name our models in this way, for example `models/braintree/base/base_payments.sql`.

We realized that while the reasons behind this convention were valid, the naming was an opinion, so in our recent update to the [best practices 3.8k](#), we took the mention of base models out. Instead, we replaced it with the principles of “renaming and recasting once” and “limiting the dependencies on raw data”.

That being said, in our dbt projects every source flows through exactly one model of the following form:

```
with source as (  
  
    select * from {{ source('braintree', 'payments') }}  
  
),  
  
renamed as (  
  
    select  
        id as payment_id,  
        order_id,  
        convert_timezone('America/New_York', 'UTC', createdat) as created_at,  
        ...  
  
    from source  
  
)  
  
select * from renamed
```

We still refer to this a base transformation. If your source data is in good shape, this transformation may be all that’s required to build a staging model, and our staging model is this SQL.

However, building a staging model may warrant several models’ worth of cleaning, correcting, and categorizing, or may require a join or union to another source. To ensure our data source flows through a base transformation, we extend our DAG upstream of the staging model, by creating a separate base model, that we then select from.

[27_AM2950x742 174 KB](#)

In our dbt projects, we place these base models in a nested base subdirectory.

```
├── dbt_project.yml
├── models
│   ├── marts
│   ├── staging
│   └── braintree
│       ├── base
│       │   ├── base.yml
│       │   ├── base_braintree_failed_payments.sql
│       │   └── base_braintree_successful_payments.sql
│       ├── src_braintree.yml
│       ├── stg_braintree.yml
│       ├── stg_braintree_customers.sql
│       └── stg_braintree__payments.sql
```

In our projects, base models:

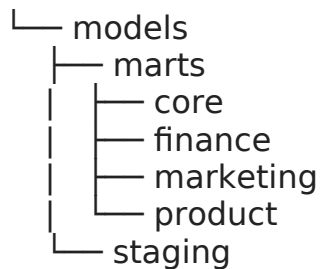
- Often use the ephemeral materialization, so they are not exposed to end users querying our warehouse.
- Are tested in a base.yml file within the same directory as the base models.

If we need additional transformations between base and staging models, we create a nested staging/<source>/intermediate directory and place these transformations in there.

Describing a business through **marts**

Marts are stores of models that describe business entities and processes. They are often grouped by business unit: marketing, finance, product. Models that are shared across an entire business are grouped in a core directory.

```
├── dbt_project.yml
```



There are entire books written on how to design models, which is beyond the scope of this article. In our view of the world, our goal is to build fact and dimension models, that are abstracted from the source data that they rely upon:

- `fct_<verb>`: A tall, narrow table representing real-world processes that have occurred or are occurring. The heart of these models is usually an immutable event stream: sessions, transactions, orders, stories, votes.
- `dim_<noun>`: A wide, short table where each row is a person, place, or thing; the ultimate source of truth when identifying and describing entities of the organization. They are mutable, though slowly changing: customers, products, candidates, buildings, employees.

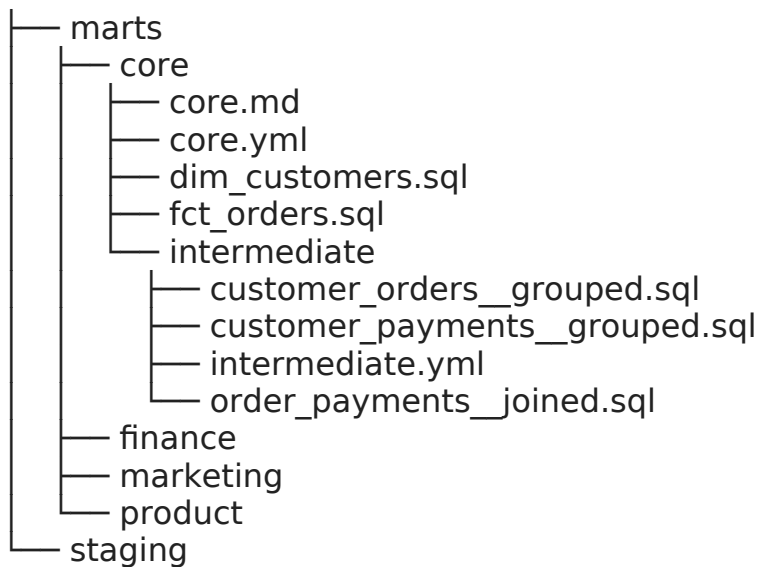
Where the work of staging models is limited to cleaning and preparing, fact tables are the product of substantive data transformation: choosing (and reducing) dimensions, date-spinning, executing business logic, and making informed, confident decisions.

This layer of modeling is considerably more complex than creating staging models, and the models we *design* are highly tailored to the analytical needs of an organization. As such, we have far less convention when it comes to these models. Some patterns we've found to be useful are:

- `fct_` and `dim_` models should be materialized as tables within a warehouse to improve query performance. As a default, we use the table materialization, and where performance requires it, we use the incremental materialization.
- Intermediate transformations required to get to a fact or dimension model are placed in a nested `marts/<mart>/intermediate` directory. They are named `<useful_name>__<transformation_in_past_tense>.sql`. The lack of prefix and use of double underscores indicates that these are intermediate models, not to be trusted, however, it may also be worth hiding these in a different [schema 582](#).
- Models are tested and documented in a `<dir_name>.yaml` file in the same directory as the models.
- Any extra documentation in a [docs block 201](#) is placed in a `<dir_name>.md` file in the same directory.

A marts directory may therefore end up looking like:





This entire project results in the following DAG:

[12_AM3782x882 269 KB](#)

Accessories to data

There are other kinds of SQL files that find their way into robust dbt projects. In addition to staging and marts, we find ourselves with model directories such as:

- **utils:** An `all_days` table. This is useful everywhere, though it never forms the basis for analysis/reporting.
- **lookups:** A user-mapping table, a zipcode-country table, etc. These are as likely to be [CSV seeds](#) as tables in a production database. You may reference it at several unpredictable points throughout modeling, and maybe even in a BI tool.
- **admin:** Audit logs, warehouse operations, Redshift maintenance, and incremental records of the miscellaneous DDL you run to make your project run smoothly.
- **metrics:** Precisely defined measurements taken from fact tables, directly conducive to time-series reporting, and tightly structured so as to allow one-to-one comparison with goals and forecasting. A metrics table lives downstream of dimension and fact tables in your DAG, and it deserves special status.

- **Packages:** While not a model folder within your main project, packages that include models (like our [snowplow 263](#) package) can be configured into custom schema and materialization patterns from `dbt_project.yml`.

In projects where we find ourselves with these additional models, we often leverage [custom schemas 582](#) as directories in our warehouse, to logically group the models, choosing a schema name that matches the directory name in our dbt project.

The best solution is no solution...

Even though this is not applicable to your use case, it needs to be said here (in case others are looking at this):

The best way to solve this problem is to avoid it! *Unless* you are trying to maintain parity across both warehouses, **I strongly recommend using two separate dbt projects.**

Further, I don't recommend maintaining parity across two warehouses – “source of truth” becomes very messy quickly!

We sometimes see this need arise when someone is maintaining a legacy warehouse while transitioning to a new one. In this case, I'd also recommend *not* trying to share models across two warehouses, and instead trying to move your dbt project over in one go!

But, since it can't be avoided in this case, it's worth sharing some wisdom we've



collected when building for compatibility across multiple warehouses . This advice will also likely be useful for anyone trying to build [packages 17](#)!

Challenges when modeling on different warehouses

You'll likely encounter the following challenges:

1. The SQL syntaxes vary slightly across warehouses
2. Your data may be loaded in a different structure
3. A performant model design varies across warehouses
4. Performance levers in dbt vary across warehouses

1. The SQL syntaxes vary slightly across warehouses

Why is this an issue?

Some of the non-ANSI functions vary across warehouses.

For example, on Redshift, to calculate the difference between two dates, you might do:

```
datediff('day', first_date, second_date)
```

But on BigQuery, you need to use a slightly different function, *and* reverse the order of



your args

```
datetime_diff(second_date, first_date, 'day')
```

And on postgres this function doesn't exist at all. Bonkers!

Using dbt-utils / adapter macros for cross-database SQL functions

Fortunately, you can use [macros 11](#) to handle the logic of which SQL function you should use. Instead of writing the SQL, you can call a macro which returns the correct SQL for your warehouse, like this:

```
{{ datediff("first_date", "second_date", "'day'") }}
```

^ This assumes that you have a `datediff` macro that would return the correct SQL for your warehouse.

Even more fortunately, we've written a whole bunch of cross-database compatible macros in our [dbt-utils 36](#) package! (Just quietly, we think this is the best repository of knowledge around the syntactical differences between warehouses out there.)

You can leverage this work by [installing 4](#) `dbt-utils` as a package, and then using the `utils` functions whenever you have SQL that needs to differ across your warehouses.

If there is a specific SQL function that we haven't written a macro for, we welcome PRs! Or you can also add your own macro to your project, following the `adapter_macro` pattern we use in `dbt-utils`.

One thing to note though is that there's a trade-off to be aware of here: using macros can reduce the readability of your models, and quoting strings correctly can be confusing (see above), so be mindful of using them too much.

2. Your data may be loaded in a different structure

Why is this an issue?

If you use off-the-shelf products (Stitch, Fivetran) to load data into your warehouse, these services may load the data slightly differently. I have more experience with Stitch, so I'll speak to the differences they have across warehouses as an example:

- Stitch loads data into BQ in an “append only” fashion
- Stitch unpacks/flattens JSON objects on Redshift (since Redshift doesn’t handle JSON well) but may not on other warehouses
- Timestamp types might differ across warehouses

Using Jinja directly in your model

If a column name, or type, varies slightly across your two warehouses, I’d handle this with an if statement in your model:

```
select
{% if target.type == 'redshift' %}
    my_redshift_column as my_column,
{% elif target.type == 'bigquery' %}
    my_bq_column as my_column,
{% else %}
    {{ exceptions.raise_compiler_error(target.type ~ " not supported in this project") }}
{% endif %}
    my_other_column
```

from my_table

On the Stitch front, [@dylanbaker](#) wrote some macros to handle the different way Stitch loads data in each warehouse, check them out [here 8](#).

Using separate models with different enabled options

If there is a huge structural difference between the data that gets loaded into each of your warehouses, it *may* be worth splitting the SQL for the models into *two separate* files.

A handy tip here is that you *can* have two models with the same name, so long as only one of them is enabled. A pattern we’ve started using lately is to use a jinja variable to determine whether a model is enabled or not, as seen [here 17](#).

You might create a directory of “staging” models for each warehouse (see [this article 10](#) for more details on how we structure projects):

```
├── dbt_project.yml
├── models
│   ├── staging
│   │   ├── braintree
│   │   │   ├── bigquery
│   │   │   │   ├── stg_braintree__customers.sql
│   │   │   └── redshift
```

└─ stg_braintree__customers.sql

Then in each model define the following [1]:

```
{{
  config(
    enabled = target.type == 'redshift'
  )
}}
```

Using adapter macros (not recommended)

Most of our packages use a pattern where each model calls a macro, and then the macro uses the adapter pattern to choose which SQL to use (for example, our [Snowplow 11](#) package).

However, we've found that this *severely limits* the readability of your code, so I'd advise against following this pattern. We are probably going to move away from it in future iterations of these packages!

3. A performant model design varies across warehouses

Credit to [@josh](#) for pointing this out.

Why is this an issue?

Different warehouses favor different model designs – for example, BigQuery pushes in favor of denormalization, so you should leverage nested objects rather than joins. Redshift doesn't easily unnest objects, so they should be flattened first.

If you want to maintain parity between your two warehouses, you'll have to choose which warehouse you'll design for and make it the first class citizen, and then just squish your other warehouse to work with that data structure.

4. Performance levers in dbt vary across warehouses

Why is this an issue?

The way to make models performant varies:

- Redshift uses sort and dist keys on tables
- BigQuery uses date partitioning

- Snowflake works the way you expect without any additional configuration

As such, the configs you supply to your dbt model will differ. For example, a Redshift model might have:

```
{{
  config(
    materialized='table',
    sort='created_at',
    dist='user_id',
  )
}}
```

Whereas a BigQuery model might have:

```
{{
  config(
    materialized='table',
    partition_by='date(created_at)',
    cluster_by='user_id'
  )
}}
```

Supplying all the configs to shared models

Since dbt doesn't mind if you supply a config that it doesn't recognize, I would just add *all the things* (i.e. performance configs):

```
{{
  config(
    materialized='table',
    sort='created_at',
    dist='user_id',
    partition_by='date(created_at)',
    cluster_by='user_id'
  )
}}
```

[1] In an ideal world, you would be able to enable your projects in your dbt_project.yml file:

```
my_project:
  staging:
    braintree:
```

```
redshift:
  enabled: "target.type == 'redshift'"
bigquery:
  enabled: "target.type == 'bigquery'"
```

However, because when dbt parses the dbt_project.yml file it doesn't yet know which profile to use (since the profile info is stored in that file!), it also doesn't know about your target.type so won't be able to parse this correctly.

Despite all the ELT tools available out there, businesses still struggle with data modeling. Data Build Tool(dbt) is a robust, **open-source tool based on SQL** that changes how organizations write, test, and deploy data transformations.

We think empowering analysts to own the data transformation pipeline is the only way to build a productive analytics team at scale. dbt makes it easier to do data modeling the right way, and harder to do it the wrong way.

James Densmore, HubSpot, Director of Data Infrastructure

It is specifically designed for **data analysts, data engineers, and data scientists** who work with large amounts of data stored in data warehouses and other data storage systems. This article will provide a deep understanding to the answer of question what is dbt data build tool, explaining what it is, how it works, and why it is becoming increasingly popular among data professionals.

What is Data Build Tool(dbt)?

[Image Source](#)

dbt is a transformation workflow that allows any business analyst comfortable with SQL to design and implement their own [data transformations](#). Eliminating the dependency on the engineering teams for making changes in the pipelines, dbt allows analysts to collaborate on data models & deploy analytics code with software engineering's best practices such as **modularity, portability, CI/CD, and documentation**. With features like version control in

Data Build Tool, you can first test your analytics code in the development before deploying them to a production environment.

Looking for an automated data pipeline platform that can be easily integrated with dbt? [Hevo](#) is the only real-time ELT No-code Data Pipeline platform that cost-effectively automates data pipelines that are flexible to your needs.

GET YOUR FREE TRIAL RIGHT AWAY!

[Image Source](#)

Core Principles of Dbt

The data building tool does data transformation and modeling based on certain core principles such as:

- **Data Warehouse centric:** Once the raw data is ingested into the data warehouse, dbt uses its in-database transformation capabilities for heavy computations
- **ELT Workflow:** Dbt does ELT on databases wherein data is first loaded and then transformed.
- **SQL-based:** It uses SQL as its domain-specific language. This enables direct transformation instead of dependence on externally controlled transformation languages or ELT tools based on GUIs.

- **Git-based version control:** Usually Git is involved in version control of dbt projects. This enables collaboration using pull requests and branch-based development.
- **Data Testing:** It enables custom data tests in SQL to be written for specific constraints. Its Schema test ensures data integrity.
- **Model Dependencies:** dbt models can refer to other models using the ref function. This allows the formation of DAG(Directed Acyclic Graph) of dependencies, which can be used to run models in the right order.

DBT Architecture

Dbt uses SQL and Jinja2 to transform and model data. It is a command-line tool with a unique architecture having the following components:

- **Command Line Interface:** This enables you to run commands for transforming, testing, and documentation.
- **Projects:** It is the foundation of DBT, which consists of models, tests, snapshots, etc.
- **Models:** These are the SQL files that show transformation logic.
- **Tests:** dbt supports both built-in and custom testing to ensure data integrity. The below query restricts the bk_source_driver field from model “fact_interaction” to not have a NULL value of more than 5% of its values set.

[Image Source](#)

- **Auto-generation:** It automatically generates a web-based documentation portal to visualize model metadata and lineage.
- **In-database computation:** The dbt tool has the capability to run SQL directly in the targeted data warehouse.

[Image Source](#)

The image shows the data lineage of some tables in the DBT user interface. This helps in understanding the flow of data from source to target.

Also, you can create a project structure by just running “dbt init” in CLI. This initializes a project and ensures all the team members work with the same template.

How does dbt work?

With the advent of ETL technology, where you first load the raw data to your warehouse and then transform, no-code automated tools like Hevo Data and Stitch Data completely automated the process. They allow you to simply replicate raw data from multiple sources to your **central repository**. However, data modeling(transformation) remains a big problem.

Businesses opt to go for a custom code using Airflow. This method is often inaccessible as it is totally written in Python and requires a large infrastructure. Or, they go for **GUI(Graphical user interface)** modeling with tools like Looker. These tools generally come with huge licensing fees and create a host of maintenance issues. This is where Data Build Tool comes in and acts as an orchestration layer on top of your data warehouse. The basic processes involved in dbt are as follows:

- **Development:** Using simple SQL SELECT statements, you can write modular transformations, build tables and views, and run models in order. You can also use python packages for complex analysis.
- **Testing and Documentation:** As you develop a model, you can first test it before sending it to production. Data Build Tool also dynamically generates detailed documentation consisting of validated assumptions, dependency graphs, and dynamic data dictionaries. You can quickly

share these with all the stakeholders, promoting more data transparency.

- **Deployment with Version Control:** Deploy your code by going through the dev, stage, and prod environments. Enjoy Git-enabled version control to return to previous states. You can also have better visibility over your transformation workflows with in-app scheduling, logging, and alerting.

How dbt differs from Other Transformation Tools?

There is no single data transformation tool like dbt that does it all. Let's see how Data Build Tool differs from other complementary tools in the market, such as Airflow, Great Expectations, and Flyway.

- Like Data Build Tool, **Airflow** helps create automated operations, run commands, and integrate with a data warehouse. Though it is not meant for full querying. The dbt platform offers a fleshed-out front-end interface for query development and coding, compared to Airflow, which is designed to focus more on the actual flow of data in its interface.
- **Flyways** assists in version control by tracking changes made to tables in the data warehouse. Unlike Data Build Tool, which manages documentation via integrations with GitHub and DevOps, Flyway needs a separate environment.
- **Great Expectations** allows you to design tests that run against your database. However, unlike Data Build Tool, it doesn't allow for integrations with [ETL](#) features as well as any editing of the actual database.

How Can I Get Started with Data Build Tool (dbt)?

Before studying Data Build Tool (DBT), we recommend the following three prerequisites:

1. **SQL:** Because dbt employs SQL as its primary language to accomplish modifications, you must be skilled in SQL SELECT statements. If you don't have this background, there are many online courses available, so look for one that will provide you with the foundation you need to begin learning dbt.
2. **Modeling:** Data modeling, like any other data transformation tool, requires a plan. This will be crucial for code reuse, drill-down, and performance optimization. Don't merely follow your data sources'

models; we advocate translating data into the business's language and structure. Modeling is vital for structuring your project and achieving long-term success.

3. **Git:** If you want to learn how to utilize dbt Core, you will need to be familiar with Git. We recommend taking a course that covers Git Workflow, Git Branching, and team collaboration. There are many fantastic options accessible online, so explore and select one that you prefer.

How does dbt integrate with Platforms for Models?

Data Building Tool is a transformation tool that allows you to view and consume the data loaded to your destination tables in a suitable form supported for analytics. Data Building tools can be integrated with various platforms such as Hevo Data for models.

Let's look at how to integrate dbt with the Hevo workflow

1. Connect the dbt projects you have created in any Git repository and run the dbt models on your destination data. You can configure dbt projects in destination types such as PostgreSQL, BigQuery, Amazon Redshift, Snowflake, and Data Bricks.
2. Schedule your dbt project for
 - Running all the **models** together
 - Running all models **individually**
 - Triggering when any event is loaded to the **destination** table
3. After completing the activity log, integrate your dbt project with Hevo Workflow.

[Image Source](#)[Image Source](#)

Check out our documentation to [learn more about dbt models](#).

The dbt project consists of resources such as models, seeds, snapshots, docs, etc.

Data Build Tool and Data Pipeline – What to expect?

The Data Build Tool takes complete care of both data modeling and testing. You can easily leverage Data Build Tool's features in your ELT pipeline as it allows you to do the following:

- With Data Build Tool, you don't need much coding experience as it allows you to perform custom transformations using simple **SQL SELECT statements**.
- dbt Cloud offers you continuous integration where you only need to push the components that change rather than the entire repository when there are necessary changes to deploy. The seamless integration with GitHub provides complete automation of your continuous integration pipelines.
- You can write **Macros in Jinja** as a reusable code that can be referred to multiple times.
- As the documentation gets automatically generated, dbt **creates lineage graphs** of the data pipeline, showing what the data is describing and how it maps to business logic.
- dbt gives you the power to **schedule production refreshes** at whatever sequence your business needs.
- You get a whole bunch of **prebuilt testing modules** in Data Build Tool, including unique, not null, referential integrity, and accepted value testing. Carrying out a test becomes completely effortless as you can reference the test under the same YAML file used for documentation for a given table or schema.

Benefits of the dbt Tool

dbt offers a complete set of eye-catching advantages that make it a powerful data transformation tool:

- You don't need to write the **boilerplate code** to create tables and views. dbt wraps your select statements in another statement at the backend to create a new relation or update an existing one.
- dbt uses **Jinja**, a lightweight templating language. It allows you to use control structures like For Loops in your queries. It also enables repeated SQL to be shared through macros.
- Using the **ref** function, you get complete control over the order of execution of your model.
- For cases when you want to reconstruct past values, dbt offers a mechanism to **snapshot** raw data for a point in time.

- You get **Git-enabled version control** and **auto-generated documentation** of your models that can be easily shared with all the stakeholders.
- After you have designed and executed tests, dbt provides assertions about the test results generated by a model to improve the integrity of SQL.
- All data transformations can be arranged with the help of dbt tool into **discrete data models**. dbt allows you to convert raw data into target datasets, and organize and materialize frequently used business logic in a fast way.
- dbt data build tool has a feature that **automates the scheduling of production refreshes** at your frequency. It also provides various ways to create and enforce data quality checks. You can create **data integrity checks** while creating documentation for any given model. Further, it has a function to make custom data tests driven by business logic.
- Testing **data integrity becomes easier** with dbt. By combining Jinja with SQL, it is easy to turn your dbt project into a programming environment for SQL. Application of a test on a given column can be made by dbt by simply referencing it under the same YAML file.
- The Data Build tool behaves like an orchestration layer on top of your data warehouse. This improves and accelerates the data transformation and integration process. It pushes code to do all the calculations at the database level. This enables faster, more secure, and easier to maintain the transformation process.
- dbt is open-source and offers a large library of reference documents, installation guides, and FAQs. Additionally, it also provides access to dbt packages. This enables you access to libraries of models and macros containing specific problems that have already been answered.

Disadvantages of the dbt Tool

Now, let's check out a few of the limitations of dbt:

- You still need data integration tools to extract and load data from multiple sources to your data warehouse, as dbt only takes care of the transformation part in ELT.
- Compared to tools offering a GUI, dbt can seem less user-friendly as it is **SQL based**.
- For cases when you need to make changes to the **boilerplate code** at the backend, you need sufficient technical expertise.

- To keep the data transformation process as readable as possible in the UI, your data engineers need to keep it clean and comprehensible.

dbt and Modern Data Stack: 5 Core Use Cases

dbt has become a popular data transformation tool that fits nicely with the **current cloud-based modern data stack**. Simplifying the data transformation, testing, and deployment process, Data Build Tool offers higher flexibility for all businesses as it offers support for multiple data warehouses:

- [dbt Redshift](#)
- [dbt BigQuery](#)
- [dbt Snowflake](#)
- [dbt Databricks](#)
- dbt Postgres
- dbt Azure Synapse

It can run on top of any of the above data warehouses and can be effectively used for several business use cases such as:

- With the ability to run tests to validate data and track the lineage of data to understand how it has been **transformed** over time, you can ensure data quality and integrity using dbt.
- Following the software development best practices, dbt provides a **consistent and standardized approach** to data transformation and analysis.
- Enhancing collaboration and communication across teams, Data Build Tool allows analysts and engineers to work together on the **same complex data models**.
- Allowing you to have a **scalable data infrastructure**, dbt can be used to define data models using SQL and then generate optimized SQL for your data warehouse.
- dbt offers a consistent and **standardized method** for data transformations and analysis, making it easier for analysts and engineers to deal with it. This can help organizations enhance the quality and dependability of their data, making it simpler to extract insights and make business choices.

dbt Best Practices

There are a few best practices suggested by Data Build Tool you can follow for the best data transformation experience:

- While managing your dbt projects in version control, it is recommended that all code changes should be reviewed in a Pull Request before merging into **master**. Go for a **dev** target when running dbt from your command line and only run against a **prod** target when executing from a production deployment.
- To maintain readability for multiple users in a dbt project, it is a good practice to have a well-defined SQL style guide.
- Instead of using the direct relation reference, you can use a **ref** function when selecting from another model.
- As your raw data structure might change over time, it is suggested to reference raw data in only one place to easily make updates to your models when required.
- Renamed the field and tables to naming conventions you wish to use for analytics.
- Instead of having multiple common table expressions(CTEs) for complex projects, you can separate these CTEs into separate models that build on top of each other in dbt.
- You can group your models in directories, allowing you to easily run subsections of your DAG and communicate modeling steps to collaborators.
- Add tests to your projects to ensure the SQL is transforming the data as you expect.

dbt Cloud vs dbt Core

[Image Source](#)

You might have seen 2 Data Build Tool products, i.e., dbt Cloud and dbt Core. But what's the difference between them? Let's have a closer look at the differences:

- dbt Core is a **free, open-source, command-line tool** that enables users to design their data models using SQL. It then converts these models into optimized SQL code that can be executed on data warehouses or other data storage systems.
- In contrast, dbt Cloud is a **cloud-based solution that offers additional features** and capabilities in addition to those offered by dbt Core. It provides a web interface for managing data models and also includes scheduling options, collaboration tools, and integrations with other data tools.

What dbt isn't?

dbt is **not a data warehouse or a database**. Instead, it is a tool that can be used in combination with a data warehouse to make its functioning easier and manage data. Also, dbt is not a programming language. It uses **programming-like syntax** to specify to load and transform data in the data warehouse. It is also not a visualization tool. However, it can be used with visualization tools like Tableau or Looker to help you understand and analyze your data.

FAQs on Data Build Tool

Here are the common FAQs on the Data Build Tool:

Who uses dbt?

dbt is used by data analysts, data engineers, and data scientists who work with data warehouses and other data storage systems. It is especially useful for organizations that have a **large number of data sources** and need to combine and transform data into a single source of truth.

Is there a cost to using dbt?

dbt Core is an open-source tool and is available for free. dbt Cloud is an enterprise-level subscription-based solution, and pricing is based on usage and features.

What kind of data storage systems does dbt support?

dbt supports various data storage systems, including data warehouses such as Snowflake, Redshift, and BigQuery, as well as databases such as PostgreSQL and MySQL.

Conclusion

Finally, you now have the complete know-how of dbt. This article provided you with comprehensive information on what is dbt in data engineering, along with its use cases. Based on your business requirements, you can choose whether to opt for dbt as a data transformation solution or go for custom coding. As dbt supports most [data warehouses](#), you can quickly onboard the tool and get started. However, there is still a gap that remains! You need to pull data from all your sources and load them into your data warehouse.

For cases when you rarely need to replicate data, your engineering team can easily do it. Though, for frequent and massive volumes of data transfers from multiple sources, your engineering team would need to constantly monitor and fix any data leaks. Or you can simply hop onto a smooth ride with cloud-based ELT solutions like Hevo Data which automates the [data integration](#) process for you and **runs your dbt projects** to transform data present in your data warehouse. At this time, the dbt Core™ on Hevo is in BETA.

[Hevo](#) is the only real-time ELT No-code [Data Pipeline](#) platform that cost-effectively automates data pipelines that are flexible to your needs. Please reach out to [Hevo Support](#) or your account executive to enable it for your team.

VISIT OUR WEBSITE TO EXPLORE HEVO

Offering 150+ plug-and-play integrations and saving countless hours of manual data cleaning & standardizing, Hevo Data also offers in-built pre-load data transformations that get it done in minutes via a simple drag-n-drop interface or your custom python scripts.

Want to take Hevo for a spin? **SIGN UP** for a 14-day free trial and simplify your data integration process. Check out the [pricing](#) details to understand which plan fulfills all your business needs.

Share your experience of learning about Data Build Tool(dbt)! Let us know in the comments section below!

[Sanchit Agarwal](#) Research Analyst, Hevo Data

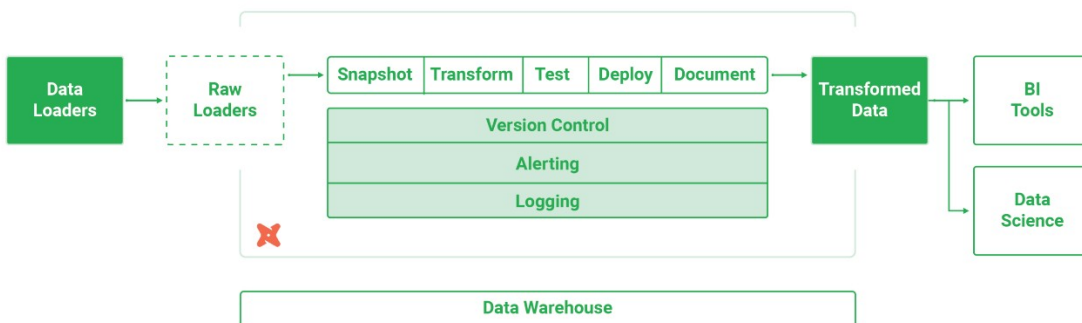
Sanchit Agarwal is an Engineer turned Data Analyst with a passion for data, software architecture and AI. He leverages his diverse technical background and 2+ years of experience to write content. He has penned over 200 articles on data integration and infrastructures, driven by a desire to empower data practitioners with practical solutions for their everyday challenges.

Data plays a crucial role in providing insights and making decisions. According to [Gartner](#), a whopping 72% of Data and Analytics leaders now have rigorous involvement with digital transformation initiatives, and cutting-edge data transformation will play a mission-critical role in these initiatives.

Point of sales (PoS) terminals, computer logs, various URLs visited, or a list of purchases made from a store – all are troves of raw data. However, raw data can only go so far in helping organizations glean information. The raw data unearthed can make sense only after ETL (Extract, Transform, Load) processes are applied and stored in databases. The data build tool (dbt) transforms your data after it is extracted and before it is loaded into databases, warehouses, or data lakes.

What is dbt?

Dbt is a tool that transforms data and has two versions, viz., the Core, a command-line interface, and Cloud, an IDE (Integrated Development Environment). It combines the robust features of a developmental framework, modular SQL, and software engineering to transform data quickly and efficiently.



The dbt tool is a boon to data analysts looking to handle data engineering activities and perform data transformations with simple SQL select statements and software coding. Multiple activities can be carried out using dbt that helps in delivering high-quality trusted data for business analysis.

Just have a brief overview of the key dbt capabilities.

Testing

dbt is a popular tool for evaluating data quality, testing integration, and checking the performance of the code. These quality parameters come as inbuilt capabilities of dbt and based upon requirements, other parameters can be programmed and run into dbt.

dbt allows QA engineers to write test programs for detecting all incomplete or non-existent entries, typical shortcomings, and mistaken values for different columns. For testing, QA specialists using dbt need to run manual testing programs first followed by automation tests. Following these tests, they can bring the required changes. Whenever a test in dbt fails, you can set the tool to send notifications for the same.

Deployment

dbt comes loaded with an inbuilt package manager for publishing data repositories whenever the data analysts and engineers need to do the same. Thanks to this ready help for deployment, data-driven projects can be easily streamlined starting from development to testing to deployment.

Documentation

Besides helping with testing and deployment, dbt offers the automatic capability to create meticulous documentation of the data-centric projects and operations carried out within any organization. dbt automatically generates visual data reports showcasing the way data is flowing and pipelined within the organizational process.

dbt users by utilizing schema files also can make documentation without much effort. All the documents created within dbt remain easily accessible and are open

for transfer to the deployment stage. Users can also see the data flow corresponding to each table within the ETL process.

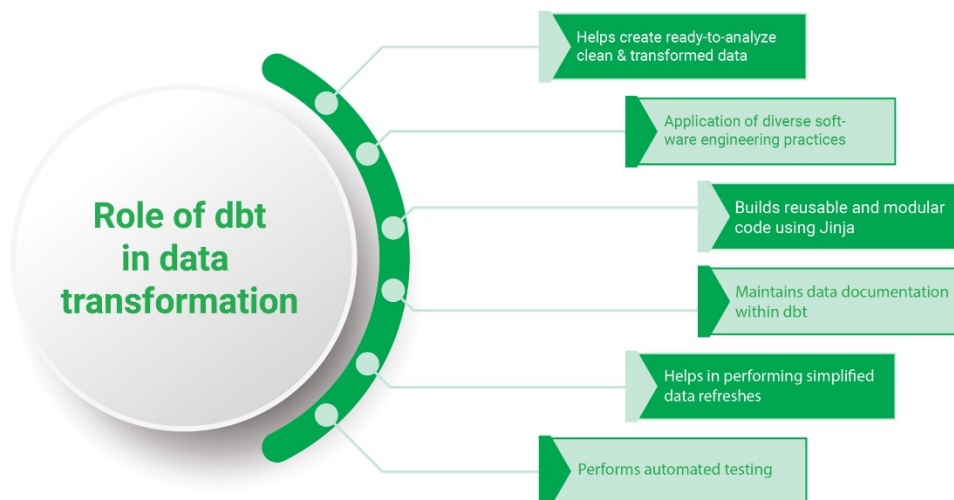
How is dbt different from other data transformation tools?

Usually, data engineers with specific skills are required to transform data with other data transformation tools. However, with dbt, data analysts with SQL knowledge can build models, write tests, and schedule jobs to produce high-quality, reliable, ready-to-consume datasets for analytics.

Dbt works on top of the data warehouse to perform transformations and integrations in a quick and fun way. The dbt tool applies the code and calculations at the database level expediting the data transformations in a simple yet secure manner.

Role of dbt in data transformation

Dbt has two main functionalities – building data models and testing the data models. It is versatile enough to connect with any modern data stack and works seamlessly with diverse cloud ecosystems – AWS, Azure, and GCP.



Clean and transformed data ready for analysis

dbt helps the data analysts write simple SQL select statements for data transformations. Without the need for boilerplate code and expertise in other programming languages, the data analysts alone can drive complex data transformations.

dbt is also known for creating data models that can easily be standardized for future uses. The data models with different business-specific and context-driven layers ensuring optimum modularity ultimately generate optimal value from the data for particular business operations and processes.

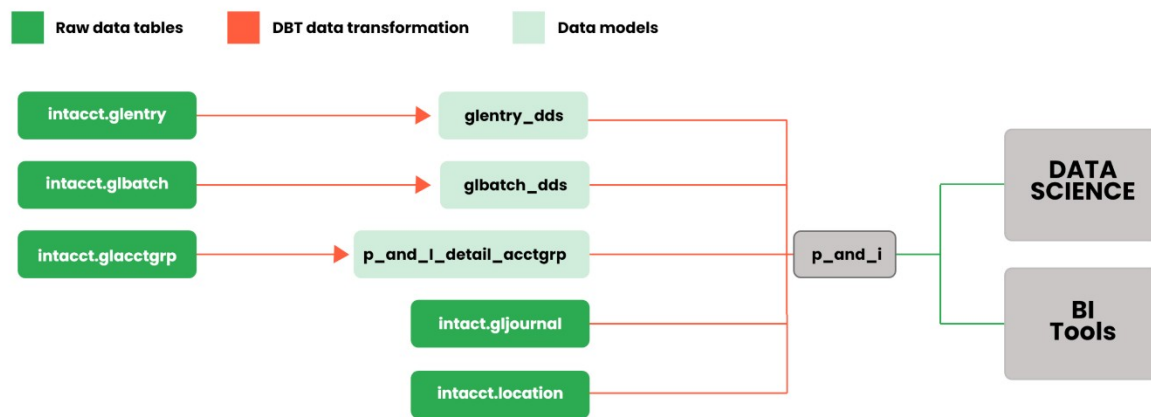
dbt for CI/CD software development practices

We can perform modular coding, version control, and testing of data and at the same time automate CI/CD to the analytics code. It allows you to test all code changes and incorporate them into production. Also, the dbt cloud is integrated

with GitHub. It means you can automate the continuous integration and do not have to manage your orchestration.

The ability of dbt in following dynamic software development practices is ensured by the way it continuously drives data Quality checks. For every given data model, dbt facilitates robust data integrity checks besides creating documentation. So, every time-continuous integration brings a new change to the data model, the snapshot table follows and documents the modifications.

Reusable and modular code with dbt data modeling



The modular approach of dbt offers the flexibility of starting from the point up to which others have contributed to the data modeling task. This component-based and modular approach saves a lot of time and effort as analysts do not need to start work on the modeling project from scratch.

Because of this flexible and layered data modeling approach of the dbt framework, the changes in the data models can be separately shared and reused as components resulting in further value additions to the work of data analysts and data scientists working with other projects.

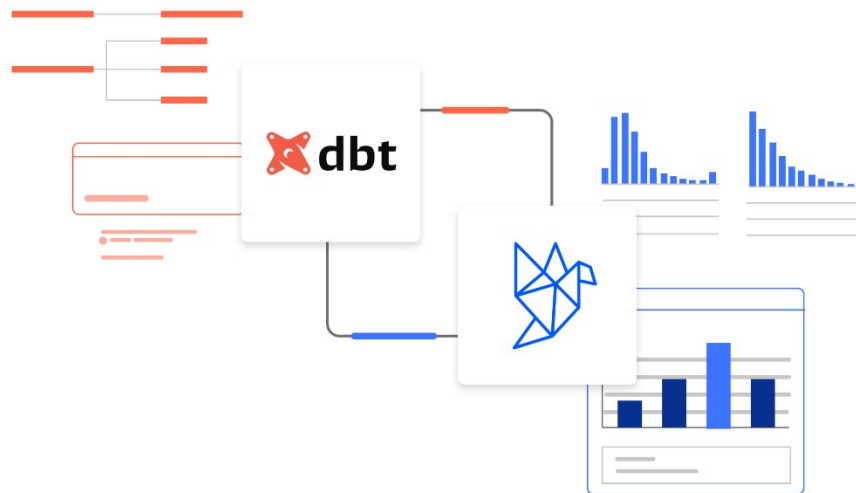
dbt for software development projects offers active help in reusing the frequently repeated code by allowing to reuse of the SQL logic in different layers of the code in a context-driven manner. Some data configuration tools with rich templates are used to ensure reusability of the code besides following other data presentations such as tables, ephemeral views like common table expressions (CTEs), and incremental views allowing inserting code at every run instance and snapshot views of the code at the start and end dates.

Maintains data documentation within dbt

These lineage graphs along with detailed descriptions ensure optimum transparency and help users to see through the data generation and the way it incorporates business logic. These lineage graphs along with descriptions for every project can be accessed through their web app.

With dbt Cloud, no separate orchestration is required. There is an integrated orchestration function that automatically generates production-ready data refreshes with the context-driven sequences required by the business.

dbt for carrying out automated testing



dbt carries out tests in two distinct ways such as schema tests and data tests. It also allows bringing together both types of tests to run data quality tests on different models. These tests can be carried out all through the model updating process without needing to depend on a separate testing framework.

On the one hand, these tests ensure updating the required changes automatically to the respective data models and on the other hand, these tests can be more meticulous in respect of updating changes than what is achieved by tests run by separate tools. Lastly, dbt automation tests can be applied to any data model. When you need to carry out testing throughout data transformation, dbt automation tests offer the best help.

Advantages of dbt

Dbt is preferred by a number of organizations to other tools simply because of the pros it offers:



In a nutshell

dbt offers absolute freedom to transform data using simple SQL select statements and manage dependencies and provides native test support while offering a surprisingly modest learning curve. It simplifies and expedites the process of data transformation by building data pipelines.

Furthermore, dbt helps in carrying out the most complex, heavy-duty transformations simply and efficiently. It provides a unique experience to data analysts and data engineers while transforming data on a single platform. It also shows detailed information about the dbt project by generating documentation along with lineage graphs for a clear analysis.

Be it data transformation, testing, and data documentation – dbt is a one-stop solution for all your data transformation needs.

Our blog on using [Airbyte with dbt](#) will help you learn more about getting started with dbt.